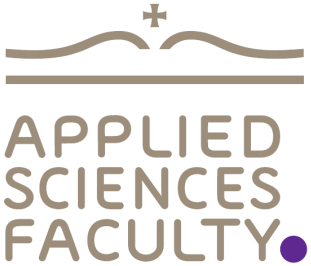
C++ — лекція 4. Вказівники та масиви.

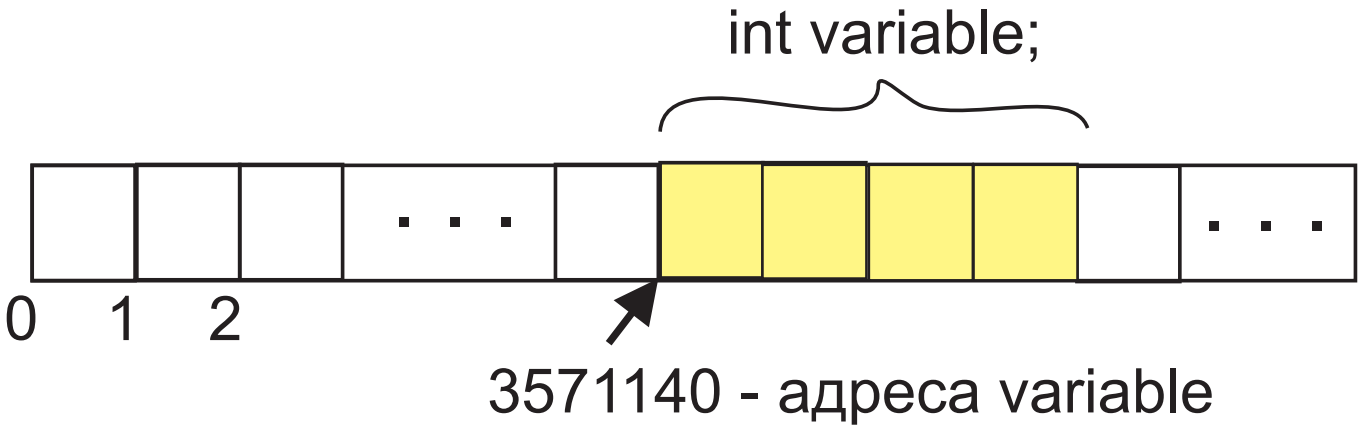




©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Адреса змінної

- Фізично, змінна — всього лиш послідовність байт в пам'яті.
- Байти — пронумеровані.
- Є сенс (і, забігаючи наперед — потреба та користь) говорити про **адресу змінної** — номер початкового її байта.



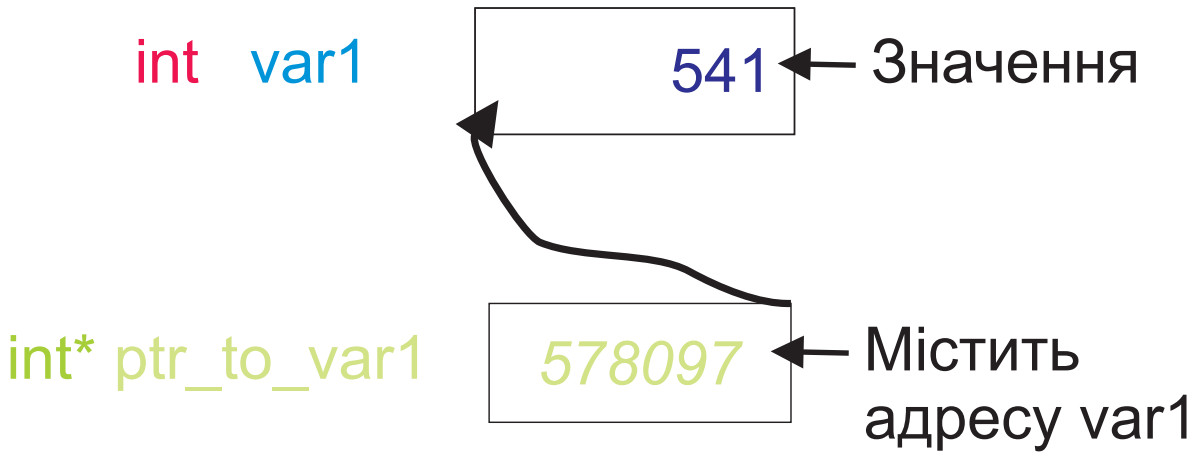
Оператор взяття адреси

&

```
int  variable ;
cout << &variable << endl;
```

у нашому прикладі виведе 3571140.

Вказівник — змінна, що містить адресу іншої змінної



- `int *ptr_i;` — вказівник на int
- `double *ptr_d;` — вказівник на double
- `double **ptr_d;` — вказівник на вказівник на double.
- `int * arr_of_p[15]` — масив із 15 вказівників на int.
- `int (*func_ptr) (double, char*)` — вказівник на функцію (назва відповідної змінної — func_ptr), яка повертає int і приймає два аргументи, типу double і вказівник на char, відповідно.

Найважливіша операція — розіменування

Розіменувати вказівник означає звернутися до області пам'яті, на яку він вказує.

Розіменування здійснюється за допомогою унарного оператора '*'.

Для визначення адреси певної змінної можна використати унарний '&'.

```
int var1;

int * pointer_to_var1 = &var1;

int var2 = *pointer_to_var1;
```

Нульовий вказівник

- Історично, вказівник, із значенням рівним нулю:

  int *ptr = 0; // *Legacy*

  інтерпретується особливим чином – це так званий **нульовий вказівник**.
- C++11 надала більш зручний варіант:

  int *ptr = nullptr;

- Воно означає, що вказівник ні на що не вказує.
- Гарантується, що жоден об'єкт не матиме такої адреси.
- Розіменовувати такий вказівник не можна – це невизначена поведінка.
- В C для нього існує макрос, NULL, рівна 0 або чомусь такому: ((void*)0), останній варіант в C++ заборонений, через суворішу систему типів. В C++ її не рекомендують використовувати, можливо, тому що вона вводить в оману, що це — щось, відмінне від банального літералу "0".
- Зауваження: бітове машинне представлення такого вказівника не обов'язково містить всі нульові біти!

Нульовий вказівник в C++11

- Число 0, як значення нульового вказівника, не дуже зручне: і виглядає, як магічна константа, і перевантаження не ті може вибирати, бо одночасно є цілим літералом. Макрос NULL, як сказано вище, ситуацію не змінює. Тому в C++11 введено нове ключове слово, `nullptr`, яке відповідає нульовому вказівнику:

```
int *ptr = nullptr;
```

- Тобто, nullptr виражає поняття "нульовий вказівник" більш безпосередньо.
- Це константа типу nullptr_t[1], який може бути перетворено у будь-який вказівник, а значення величин цього типу лише одне — nullptr.
- ◇ Підтримується також перетворення у bool, але не в цілі типи – неявного перетворення в інші типи немає.
- Перетворення будь-якого вказівника у тип nullptr_t (наприклад, з використанням static_cast<>) повертає nullptr.
- nullptr, на відміну від 0, нормально співпрацює із форвардингом аргументів.

[1] Цікаве використання цього типу – перевантаження, які не дають викликати з нульовим вказівником: "More compile-time checks with std::nullptr_t". Це не дуже сильний захист, але все ж, збільшує кількість випадків, коли проблему видно ще під час компіляції.

Вказівники без типу — void

- Вказівник спеціального виду — без інформації про тип.
- Оголошується так:

```
void *ptr_without_type;
```

- Йому можна надавати вказівники будь-якого типу.
- Однак, його не можна надавати іншим вказівникам без **явного приведення типів**, не можна розіменовувати — звертатися до пам'яті, на яку він вказує.

```
void *ptr_without_type;
double var;
double *var_ptr1 = &var;
ptr_without_type = var_ptr;

double *d_ptr5;

// Приведення в стилі C
d_ptr5 = (double *)ptr_without_type;
// або
// Приведення в стилі C++
d_ptr5 = static_cast<double *>(ptr_without_type);
```

Масив — колекція однотипних елементів



- Елементи масиву розташовуються в пам'яті послідовно, один за одним, без пропусків.
- Оголошується масив наступним чином:

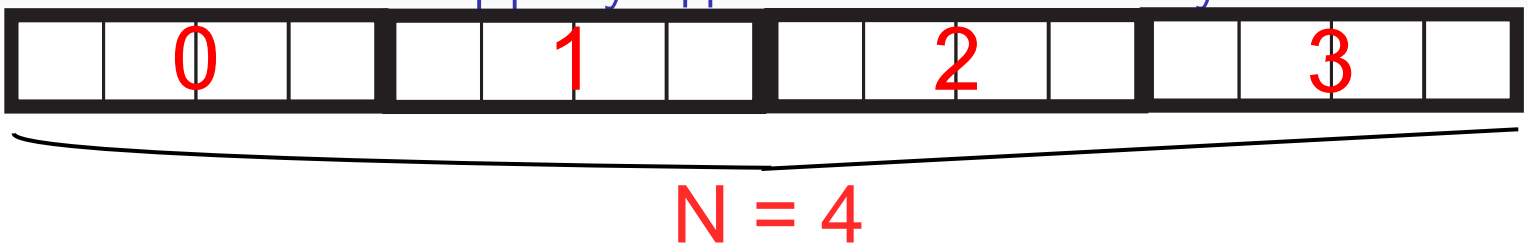
  ```
  type array_name[N];
  ```

  Наприклад,

  ```
  int arr1 [4];
  ```

  оголосить масив із 4 чисел, з номерами від 0 до 3.
- Елементи масивів у C/C++ нумеруються від нуля. Тобто, масив із N елементів матиме елементи з номерами від 0 до N-1.
- До імені масиву — ті ж вимоги, що й до імен змінних та функцій.

Доступ до елементів масиву



- Розмір так оголошеного масиву:

```
int arr1 [4];
```

  повинен бути заданим на момент компіляції[2] — повинен бути константою або константним виразом.

- Звертання до елемента з конкретним номером i, здійснюється так: array_name[i]:

```
int z, i=2;
z = arr1 [0];
z = arr1[ i ];
```

- **Перевірка виходу за межі не здійснюється!** — все на відповідальності програміста.

[2] C99 дозволив підтримку локальних масивів, розмір яких на момент компіляції не був відомий – Variable-length arrays. Однак, вони ніколи не входили в жоден стандарт C++, їх підтримка компіляторами C далеко не всезагальна, а сама концепція викликає помітну критику. Тому **не використовуйте їх, навіть якщо ваш компілятор це дозволяє**. Детальніше див. `https://en.cppreference.com/w/c/language/array`. Для GCC спосіб заборонити: Werror=vla.

- Розмір масиву дорівнює добутку розміру елемента на кількість елементів. Наприклад, для наступного масиву:

```
const size_t arr_size=5;
double arr1[arr_size];
```

матимемо

```
sizeof(arr1)=arr_size*sizeof(double)=5*8=40;
```

- **Увага,** sizeof дає розмір масиву у **байтах**.
- "Історичний" спосіб отримати кількість елементів:

```
sizeof(arr1)/sizeof(arr1[0])
```

Тобто, розмір масиву у байтах, поділити на розмір нульового[3] елемента.

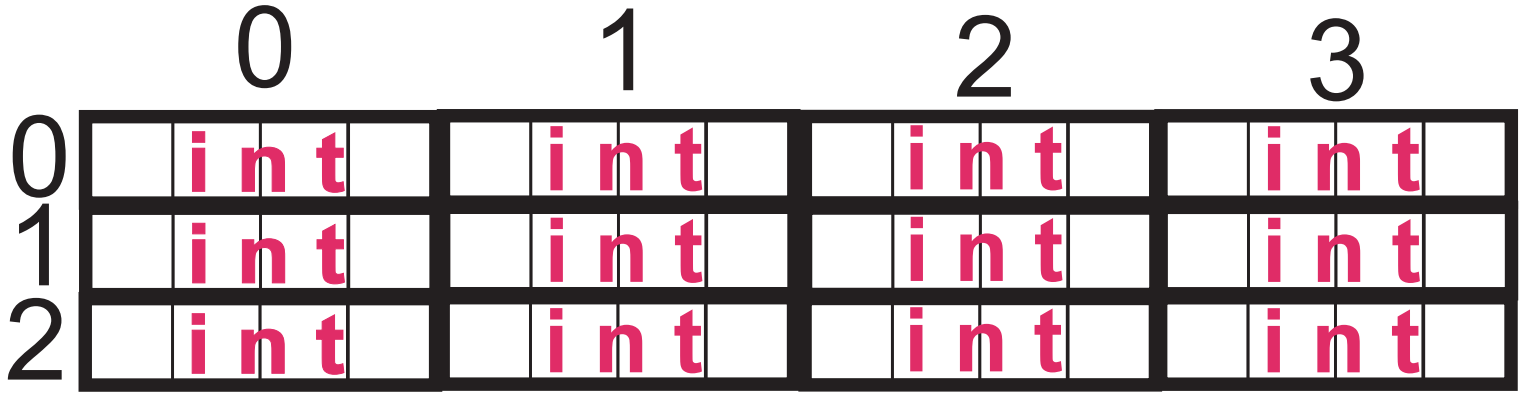
◇ Однак, в C++17 краще використовувати std::size()! – цей спосіб менш вразливий до помилок.

◇ В C2y планується додати аналогічний засіб як ключове слово.

- Увага, передачу масивів у функції буде розглянуто окремо.

[3] Взагалі, на розмір будь-якого із елементів, але нульовий присутній завжди — можна створити масив з одного елементу, але не можна з нуля елементів.

Багатовимірні масиви




```
int arr3 [3][4];
```

- Фактично, цей двомірний масив — масив із трьох масивів по чотири int. В пам'яті він так і розташовується — нульовий рядок (4 елементи), перший (4 елементи), і т.д.
- Зокрема, arr3 є вказівником на int(*)[4] – масив із чотирьох int:

```
int (*pl)[4] = arr3;
```

- Розмірність може бути більшою за 2:

```
int arrX [30][14][235][111][980];
```

◇ Вказівник буде мати тип `int (*)[14][235][111][980];`

Ілюстрація до розташування багатовимірних масивів у пам'яті

Наступний код еквівалентний:

```
const int cols=4;
const int rows=3;

int table1[rows][cols];

for (int n=0;n<rows;n++)
{
  for (int m=0;m<cols;m++)
  {
    table1[n][m]=(n+1)*(m+1);
  }
}
```

```
const int cols=4;
const int rows=3;

int table2[rows*cols];

for (int n=0;n<rows;n++)
{
  for (int m=0;m<cols;m++)
  {
    table2[n*cols+m]=(n+1)*(m+1);
  }
}
```

- "Multi-dimensional, dynamically sized arrays in C (and C++) the right way" – есей на тему.

Ініціалізація масивів

- Оголосивши масив, ми можемо зразу надати його елементам значення:

```
int arr2[5] = {10,11,55,77,1};
```

- Кількість елементів у фігурних дужках може бути меншою або рівною кількості елементів масиву.
  - ◇ Якщо елементів у **списку ініціалізації** замало, решта будуть ініціалізовані нулями або іншими, прийнятими для типу значенням по замовчуванню. Для класів викликатимуться відповідні конструктори по замовчуванню.
- Вказувати більше елементів, ніж може містити масив, заборонено.
- Також, можна робити так:

```
int arr2a[] = {10,11,55,77,1};
```

  Тоді розмір масиву буде розраховано автоматично.
- **УВАГА!** Так масиви можна тільки ініціалізувати. Не існує аналогічних операторів присвоєння.
- Див. есей "The "array size constant" antipattern".

Масиви невідомого розміру

- Ще одне використання порожніх квадратних дужок – вказівка, що розмір масиву у цій точці невідомий:

  extern int arr [];

- Для багатомірних масивів невідомою може бути лише перша розмірність:

  extern int arr2d [][4];

- Їх використання доволі обмежене.

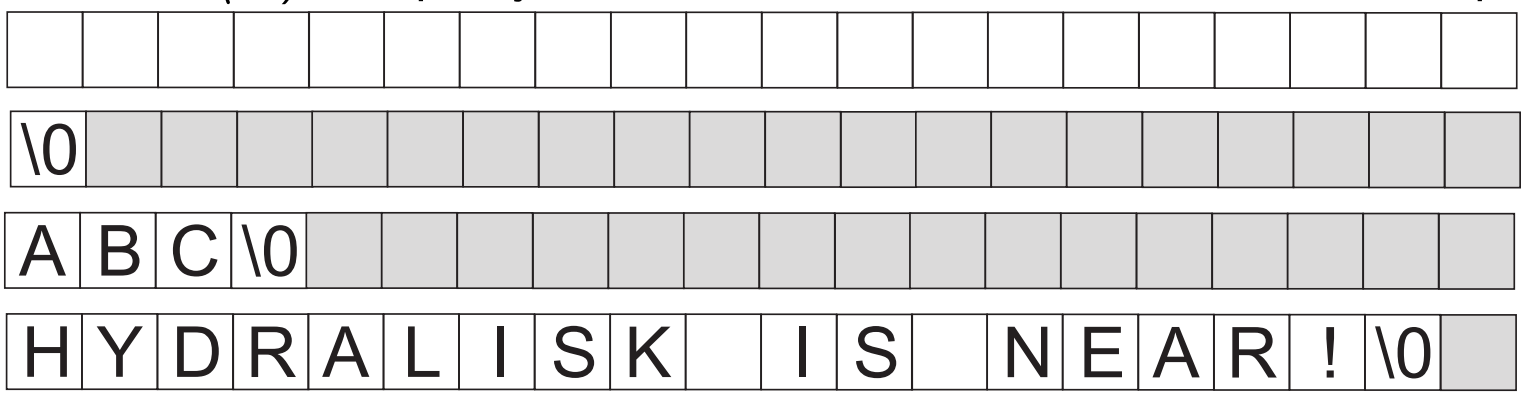
Variable-length array (VLA) – C

- В C99 додали масиви, розмір яких можна визначати під час виконання – Variable-length arrays.
- Через велику кількість негативних реакцій, їх зробили опційними в C11 (перевірити можна макросом __STDC_NO_VLA__).
  - Однак, в C23 знову зробили обов'язковими.
- В C++ такого засобу немає і (поки) не планується.
  - Однак, GCC і новіші версії Clang їх мало того що підтримують, як нестандартні розширення, так і вмикають за замовчуванням. Заборонити їх можна ключем `-Werror=vla`.

- Через спосіб, яким організовано вбудовані масиви, ar[i] є синонімом i[ar].
- Основне використання цього – запитання на співбесідах. Ймовірно – не дуже хороші запитання
- Раймонд Чен запропонував кілька більш практичних, хай і NSFW, способів: `https://devblogs.microsoft.com/oldnewthing/20230403-00/?p=108005` – "C++17 creates a practical use of the backward array index operator".
  - Перше – обійти перевантажений оператор [] для класу ar.
  - Друге – C++17 гарантує, що у виразі expr1[expr2], expr1 буде обчислено до виразу expr2, тому expr2[expr1] дозволить інвертувати цей порядок.
- Не варто використовувати ці трюки, звичайно!

Спеціалізовані масиви — С-стрічки

- З точки зору С, стрічка — послідовність символів (типу char), яка закінчується символом з кодом 0.
- Так як ввести такий символ з клавіатури неможливо, для нього існує відповідна керувальна послідовність, "\0"
- Завдяки виділеному символу кінця, стрічки, можна зберігати у масивах більшого розміру — не обов'язково заповнювати його повністю.
- Наприклад, char[20], може зберігати і порожню стрічку (тільки символ \0), і стрічку з однієї, двох, і більше, аж до 19, літер.



- Стрічкові літерали мають тип const char[X], але, для зворотної сумісності, їх можна присвоювати типу char*. (C++11 нарешті заборонив це.) Однак змінювати такі літерали не можна! — це приводить до невизначеної поведінки (undefined behavior).

Масиви і вказівники

- Змінна-масив насправді зберігає вказівник на початок масиву, або, іншими словами — є вказівником на його початковий елемент.

```
unsigned int array33 [15];
cout << "array33 : " << array33 << endl;
cout << "&array33[0]: " << &array33[0] << endl;
```

```
array33 : 0031F8F0
&array33: 0031F8F0
```

  Єдина відмінність — змінна-масив є константою, змінювати її не можна: ~~`array33=some_ptr;`~~
- Гарантується, що вказівник на елемент після останнього — коректний. Він використовується в багатьох алгоритмах. Однак, розіменовувати такий вказівник заборонено — відповідного елемента не існує!
- Спорідненість вказівників та масивів йде ще далі — до елементів масиву, на які є лише вказівник, можна звертатися за допомогою звичного синтаксису: `d_ptr[i]`, що є еквівалентно `*(d_ptr+i)`. Останній вираз містить деякі хитрощі, пов'язані із **арифметикою вказівників**.

Арифметика вказівників

- Так як вказівники можуть вказувати не лише на окремі змінні, але й на області пам'яті із послідовними однотипними змінними — масиви, природно, що крім ініціалізації та присвоєння, вказівники можна переміщати — змушувати вказувати на інший елемент (в коректному коді — того ж!) масиву.
- Для цього використовується *арифметика вказівників*.
- Додавання одиниці до вказівника пересуває його так, що він вказуватиме на наступний елемент свого типу, віднімання одиниці — на попередній[4].
- Тобто, `ptr+1` збільшує адресу, яку містить вказівник, на `sizeof(ptr)`.
- Можна додавати чи віднімати й відмінні від одиниці цілі значення. При цьому вказівник переміщається на відповідну кількість елементів вперед або назад.
- Різниця двох вказівників на один масив, повертає кількість елементів між ними. Увага — результат віднімання вказівників на різні масиви невизначений! Додавання вказівників позбавлене очевидного сенсу, тому заборонене.

[4] Можна використовувати як `++ptr`, так і `ptr+1`.

Посилання (lvalue references)

- Lvalue посилання — синонім певної змінної, її альтернативне ім'я.
- Оголошується посилання за допомогою оператору оголошення, "&". Не плутайте із іншими використаннями цього символу!
- Посилання слід зразу ініціалізувати. Змінювати потім, синонімом якої саме змінної воно є, не можна.
- Посилання повинне мати той же тип, що і змінна, на яку воно посилається — ніякого приведення типів не може відбуватися!
- Приклад:

```
int var = 5;
int &ref = var;
cout << var << ", " << ref << endl;
ref = 3;
cout << var << ", " << ref << endl;
var = 7;
cout << var << ", " << ref << endl;
```

- Часто компілятор реалізовує посилання як константні вказівники, що автоматично розіменовуються, однак на практиці це знання рідко буває корисним[5].

[5] Наприклад, достатньо розумний компілятор часто може взагалі не створювати якісь реальні об'єкти для посилань.

Посилання на константи

- "Звичайні", не-константні, посилання, слід ініціалізувати lvalue — щоб існувала пам'ять, на яку вони посилаються. Тому, наприклад, надавати їм літерали не можна!
- Однак, константні посилання, наприклад `const int &`, можна ініціалізувати не тільки lvalue, але й rvalue, або просто літералом. Крім того, навіть тип не обов'язково повинен співпадати, якщо існують відповідні неявні перетворення типів.
- У описаному вище випадку створюється тимчасова, невидима програмісту, змінна, яка і служить ініціалізатором для посилання. За потреби виконуються необхідні перетворення типів.
- Наприклад, така конструкція:

  double &ref2 = 5.3;

  заборонена, а така:

  const double &ref2 = 5.3;

  дозволена. Це полегшує використання ідіоми оптимізації передачі значення у функцію за допомогою константного посилання.
- Причина існування такого винятку – ідіома передачі аргументу функції за константним посиланням.

Dangling reference та продовження часу життя

- Посилання не буває "нульовим" – воно завжди прив'язане до якогось об'єкту.
  - Але можна створити ситуацію, коли об'єкт, на який воно посилається, припинив існування – це так звані "висячі посилання", dangling reference. Звертання за ними – невизначена поведінка.
- Якщо посилання пов'язане із тимчасовим об'єктом або його членом, час життя об'єкту продовжується, поки існує посилання.
- Однак, із цього правила багато винятків:
  - Посилання на посилання не продовжує часу життя. Наприклад, нехай функція отримала константне посилання і повернула його ж. Якщо аргумент був тимчасовим значенням, створеним для її виклику, його буде знищено при виході з функції.
  - Тимчасове значення, що повертається із функції знищується зразу – таке посилання завжди "провисне" (Приклад такого тимчасового: return a + b;).
  - Детальніше див. `https://en.cppreference.com/w/cpp/language/reference_initialization#Lifetime_of_a_temporary` та `https://quuxplusone.github.io/blog/2020/11/16/lifetime-extension-tidbit/`.
- Як і для вказівників, не можна повертати посилання на локальні змінні функції – їх буде знищено під час виходу з неї.
- Повернення посилання на тимчасові значення з функцій – завжди dangling reference.
- Для range for звертання до під-об'єкту тимчасового об'єкту не продовжувало час життя, виправлено в C++23.
  - Нарешті, в C++26 обіцяють це заборонити, "C++26: Disallow Binding a Returned Reference to a Temporary".

Посилання

- `https://quuxplusone.github.io/blog/2021/11/09/pass-string-view-by-value/` – стаття із детальним аналізом, коли передавати за значенням.
- `https://wg21.link/P0773` – "Towards meaningful fancy pointers", цікава філософська стаття про суть вказівників.
- "Field-testing -Wassign-to-class-rvalue"
- "Improving Code Safety in C++26: Managers and Dangling References" – есей на тему dangling referencei пов'язаних з ними покращень в C++26.
- "Falsehoods programmers believe about null pointers" – дуже повчальна стаття.
  - ◇ Зокрема, нульовий вказівник не завжди фізично містить нульові біти – наведено приклади, (AMD GCN – нульовий вказівник фізично зображений як -1, оскільки нульова адреса потрібна в роботі, NVIDIA Fermi, див. також відповідь на запитання "Seriously, have any actual machines really used nonzero null pointers, or different representations for pointers to different types? ")
  - ◇ Нульовий вказівник і вказівник на нульовий байт RAM можуть інтерпретуватися по різному, навіть якщо бітові представлення однакові – починає відігравати роль походження вказівника.
- "Three Meanings of Reference" – передавання за константним посиланням, локальний синонім, член класу і вираження намірів щодо володіння.